# Introduction to XML Tutorial

*This tutorial contains only a portion of the material covered in WestLake's hands-on XML class.*

*We have included the entire table of contents of the XML workbook to give you a better feel for the contents of the full course.*

# Table of Contents

# Introduction to XML

## Overview

XML (e**X**tensible **M**arkup **L**anguage) has been touted as the language which will replace all other Web tools, making HTML, databases, and much more obsolete. Although it will not do that, it has huge advantages over current Web technologies in flexibility, portability, and data awareness. The key to these abilities is a designer's freedom to design XML tags that accurately describe the structure and content of his or her data.

## The Development of XML

XML, like HTML, grew out of the SGML language. Like HTML, it is simplified so that users have a predefined set of rules to follow. Like SGML, designers write their own tags and tag structure to accurately define their data. And unique to XML, the World Wide Web Consortium (*http://www.w3.org*) has developed rigorous rules to define well-formed XML, ensuring that XML data will be readily available to any application that needs it.

Although SGML has been in existence since 1970, and HTML since 1990, XML was not proposed until 1996. It was designed to attain the following goals (*http://www.w3.org/TR/WD-xml-961114.html#sec1.1)*:

1. XML shall be straightforwardly usable over the Internet.
2. XML shall support a wide variety of applications.
3. XML shall be compatible with SGML.
4. It shall be easy to write programs which process XML documents.
5. The number of optional features in XML is to be kept to the absolute minimum, ideally zero.
6. XML documents should be human-legible and reasonably clear.
7. The XML design should be prepared quickly.
8. The design of XML shall be formal and concise.
9. XML documents shall be easy to create.
10. Terseness is of minimal importance.

Two of these features deserve special attention, as they have had a major impact on the way XML Web documents are prepared.

- First, that XML documents be able to support a wide variety of applications. This indicates that XML should not include display information, as that information will necessarily be specific to a particular platform. A well-written XML document will be readable through the Web, by cell phones, by databases, and by many other platforms.

- Second, that XML should be straightforwardly usable over the Internet. This specification, in conjunction with that XML be required to be platform-independent, mandates that XML for the Web be transmitted along with other "helper" files which will specify how browsers should treat the XML data.

## An XML Example

Before we go further into the nature of XML, let's look at a sample document. Doing so will allow us to discuss the specifics of XML syntax more knowledgeably. This example is **simple_example.xml**:

```
<?xml version="1.0"?>

<client>
   <name>
      <firstname>John</firstname>
      <lastname>Doe</lastname>
   </name>
   <address type="home">
      <street_address>123 School St.</street_address>
      <city>Ithaca</city>
      <state>NY</state>
      <zip>14850</zip>
   </address>
   <age>26</age>
</client>
```

As you can see from the above example, XML describes the **logical** structure of data. XML tags (other than the initial language declaration) have no inherent meaning, but instead can be applied flexibly as the designer chooses. What those meanings are, and how they are translated into visual results, is determined by separate sections of the XML application. What you see above is commonly termed an **XML Datasheet**. To turn this into an XML application will require the inclusion of a couple of additional files: a Document Type Definition (DTD) or Schema, and a stylesheet, written in either Cascading Style Sheets (CSS) or Extensible Stylesheet Language (XSL). We will examine these components in upcoming sections.

## Why XML?

Even before you get started, it is worth thinking briefly about what sorts of applications find XML useful. We will see examples of these sorts of applications throughout the tutorial. These are:

### When you need to output the same data in multiple formats

As you saw above, XML holds the data of a document without specifying how it is displayed. At first, this may seem like a detriment, and you do need some sort of stylesheet to transform the raw data into something appropriate for the end display mechanism (such as HTML for a browser, formatted text for a fax or mailing, and plain text for sending out as an email). Storing the document in XML means that you only have to write the data once, and if you rewrite your document, you can just re-apply different stylesheets and have the end results automatically

generated. What's more, if you standardize your document types (such as press releases, for example) on a specific set of tags your stylesheets will be reusable without modification.

## When you need to exchange data between otherwise incompatible systems, applications, or organizations

For example, consider the case of a manufacturer. They routinely need to subcontract out to produce various parts for inclusion into their products. In the pre-XML world, they had to send out a document specifying what they needed in a bid to produce these products, and might easily receive the information back in various formats: paper documents, faxes, emails, attachments, and spreadsheets, at least. A person would have to then enter this information into some application so that the different bids could be compared. The added layer of data entry adds time, expense, and error to the process.

With XML the same manufacturer can send out their requirements in the form of a Document Type Definition (DTD) or Schema. Both of these specify a particular tag structure that is legal for a given application. Once a subcontractor receives the structure that they are expected to produce, they can tune their applications to produce XML as specified by the DTD.

## When you need to store structured data, but a database is not appropriate or necessary

For many applications, particularly applications where data is being generated in multiple locations with questionable connectivity, it can be difficult or undesirable to have team members connecting directly to the main database for a project. XML allows the opportunity to store structured information in a format that can be easily imported into a database, or evaluated without one.

## When you want multiple different views of information, presented dynamically

We will learn more about this capability later in the tutorial. For now, consider the process you need to go through to re-sort a display in a traditional database driven application. Imagine that you wish to see a display of information sorted by last name rather than zip code. You have to follow the following process:

1. Browser makes a new HTTP request to the server

   Server processes the request with a middleware application

   Middleware application builds a new SQL query for the database

   Middleware application creates a connection to the database, and passes it the SQL query

   Database receives the query and retrieves the recordset

Database returns the recordset to the middleware application

Middleware application generates the HTML from the recordset, and sends it via HTTP to the browser

Browser displays the information

Whew! That's a lot of steps for one application, and requires your server and your database to do extra work for reformatting. However, if you can provide your data in XML form to an XML-capable browser, the process is simplified to:

2. Browser uses JavaScript to modify XSL stylesheet

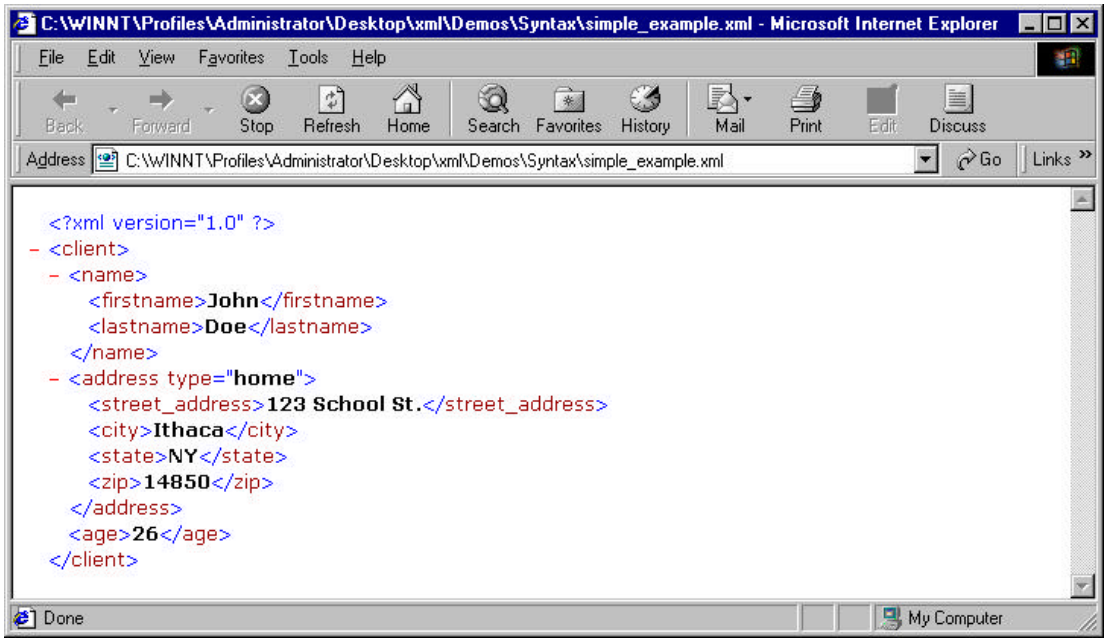
   JavaScript reapplies (now modified) XSL stylesheet to XML data

   JavaScript places the generated HTML content into the page

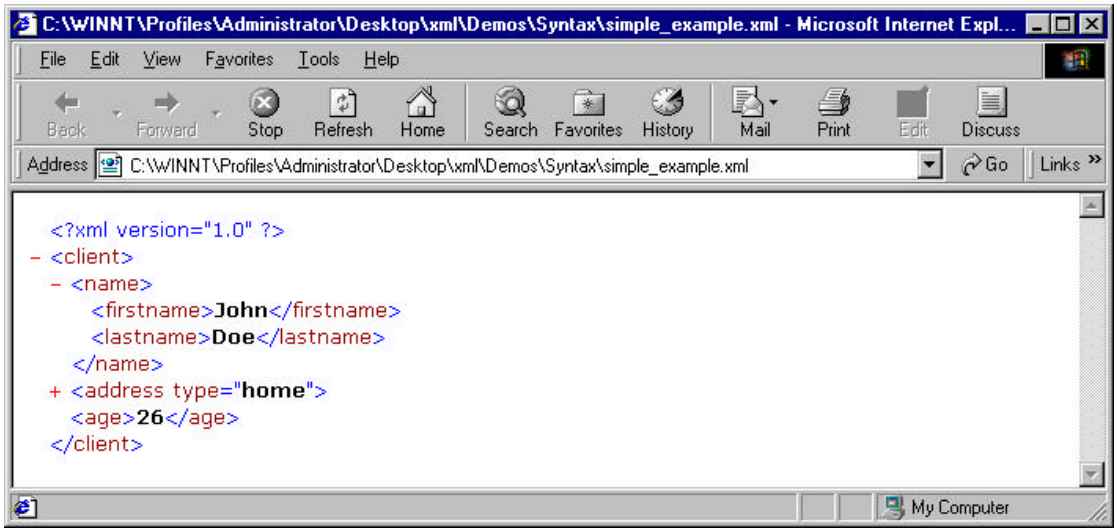
As you can see, not only is the process a much shorter one, but it also is entirely processed by the browser's processor. By taking advantage of efficiency savings like these, you can make applications that are extraordinarily responsive.

# XML and Browser Compatibility

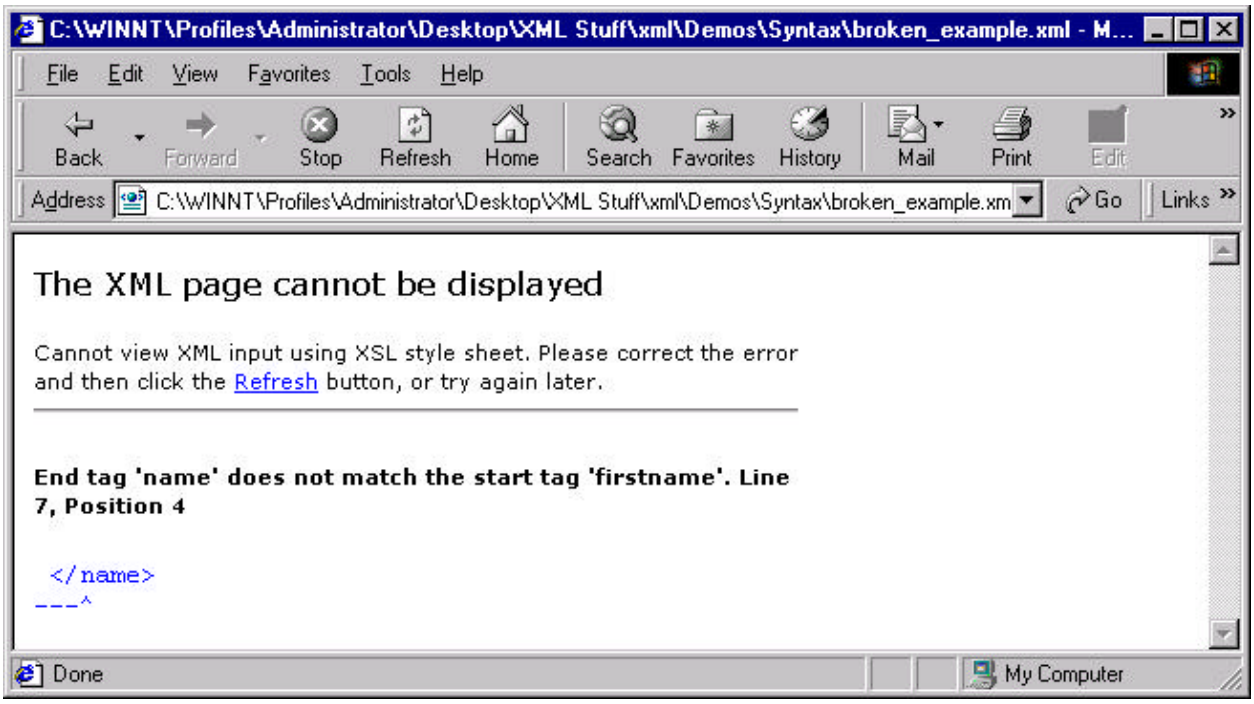
At this time, the only browser that can interpret and display XML documents is Microsoft Internet Explorer 5. IE5 has implemented the standards proposed for XML by the World Wide Web consortium, and is a stand-alone XML viewer. We will make use of these capabilities many times during this tutorial. For example, the code you saw in the previous section, viewed in IE 5, would look as follows:




Note that Internet Explorer understands the heirarchichal structure of the tags, and actually allows you to expand or contract the display to show the depth of data you choose. For example, clicking on the small "**-**" next to the **address** tag produces the following display:



Similarly, if Internet Explorer cannot parse the XML code (because it breaks some syntax rule or does not conform to a given document type) it produces an error message (see **broken_example.xml**):




Other major browsers do not fully support XML. However, Netscape 6 does support some of the published XML standards, particularly the display with Cascading Style Sheets (CSS). Subsequent releases will surely contain additional support. Internet Explorer 4 does have a limited ability to parse and display XML through a variety of plug-ins and controls, but has no native ability to do so. Netscape 4 and earlier, and IE 3 and earlier, will not recognize XML as a valid Web format, and will try to download the file instead of displaying it.

## Microsoft and XML Extensions

Microsoft has commited iteslf publicly to supporting the XML standards, as published by the W3C. In addition, they have been at the forefront of the **XML Working Group**, which proposes, evaluates, and publishes additions or changes to the XML standards. This effort has meant that Microsoft has been quite aggressive in adopting XML standards, and in producing tools with the capability of working with XML.

However, there remain three areas where Microsoft's compliance differs from the published standards. The first is in the dynamic integration of XML data into HTML and JavaScript pages. The W3C has not yet issued final standards on how this should be accomplished, and in an effort to provide developers with tools, Microsoft has instituted a few extensions to the XML specifications. Most of these extensions have been presented to the World Wide Web

Consortium for inclusion into the next draft of the XML specifications, although there is no guarantee that they will be accepted.

The second area where Microsoft's implementation is at odds with the W3C's is in the Schema proposal. Microsoft was one of many different groups who proposed a proposal for XML schemas, and they built support for their version of schemas into Internet Explorer 5. The final syntax of the XML Schemas adopted by the W3C is rather different from Microsoft's proposal (though both are widely similar in scope and approach). The W3C Schema proposal was adopted as final in May, 2001, and Microsoft has announced that it will implement support very soon. However, the current release of Internet Explorer does not yet support them. This tutorial focuses on the W3C Schema proposal.

Finally, the third area where Internet Explorer differs from the standards is in XSL support. Unfortunately, the draft of the XSL standards was in very preliminary form when Internet Explorer was developed, and the standards have since changed significantly. However, there are millions of copies of Internet Explorer with support for a now-obsolete dialect of XSL. In December, 2000, Microsoft released a new version (3.0) of its XSL parser, which is available as a free download from *http://www.microsoft.com/XML/*. The newer parser includes almost complete XSL standards support, and Microsoft has pledged to continue working on the parser until it reaches full compliance (and, in fact, as of June, 2001, Microsoft had released a technology preview of its 4.0 Parser). All the examples in this tutorial assume that the Microsoft 3.0 XSL parser has been installed.

The vast majority of this tutorial is standards compliant and platform-independent. Where we use anything that is Microsoft-specific (particularly in the dynamic XSL transformation section at the very end of the tutorial) we will note the applications as such, and discuss any possible alternatives.

## Why Use a Browser to Display XML?

Even a brief glance at the previous "Why XML" section should make it clear that the majority of XML applications are run on a server. So, why do we teach this tutorial using Internet Explorer?

3. Both Internet Explorer and the MSXML 3.0 Parser are free.

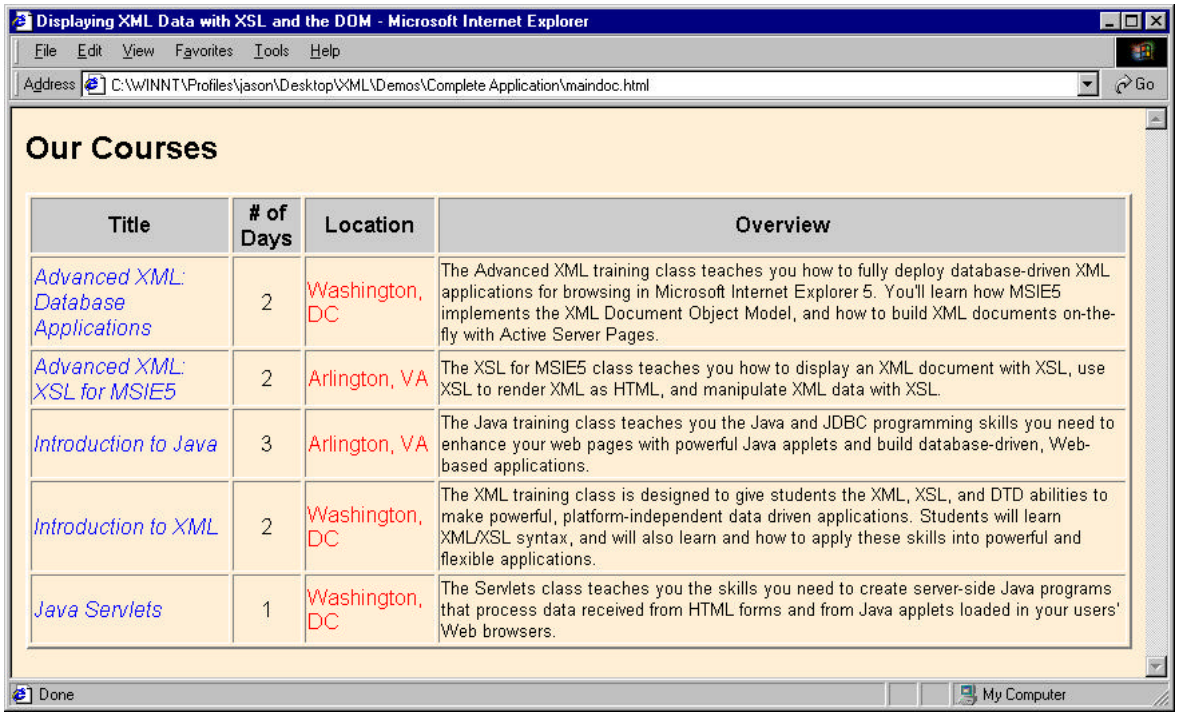
4. Internet Explorer is widely available, and the MSXML Parser is a free download from Microsoft.

5. The support for XML in the MSXML 3.0 Parser is very well developed, and provides clear, helpful error messages should you, for example, write poorly-formed XML.

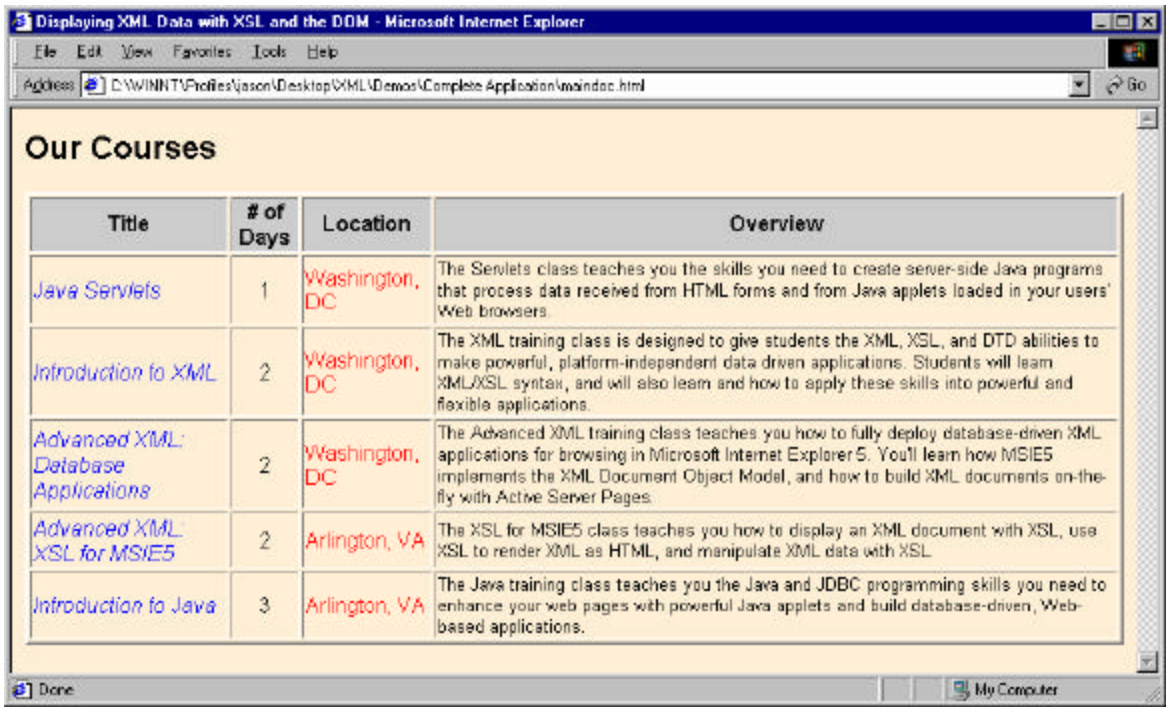
6. Professional developers writing applications that will eventually run on a server often use Internet Explorer as a lightweight environment for testing their applications because of its ease of use. Essentially, Internet Explorer makes a perfect lightweight development environment for XML.

# Examining an XML Application

The easiest way to get a feel for the different components of an XML application is to take a look at a complete one. So, let's look at our first application. Please open the file **maindoc.html** in Internet Explorer 5. You should see the following display:




At first glance, it looks like a normal tabular display of HTML data, with a little formatting and coloring thrown in. However, try clicking on the different columns. If you click on the "Location" column, the table resorts by location. If you pick the "# of Days" column, it will be resorted by class length:

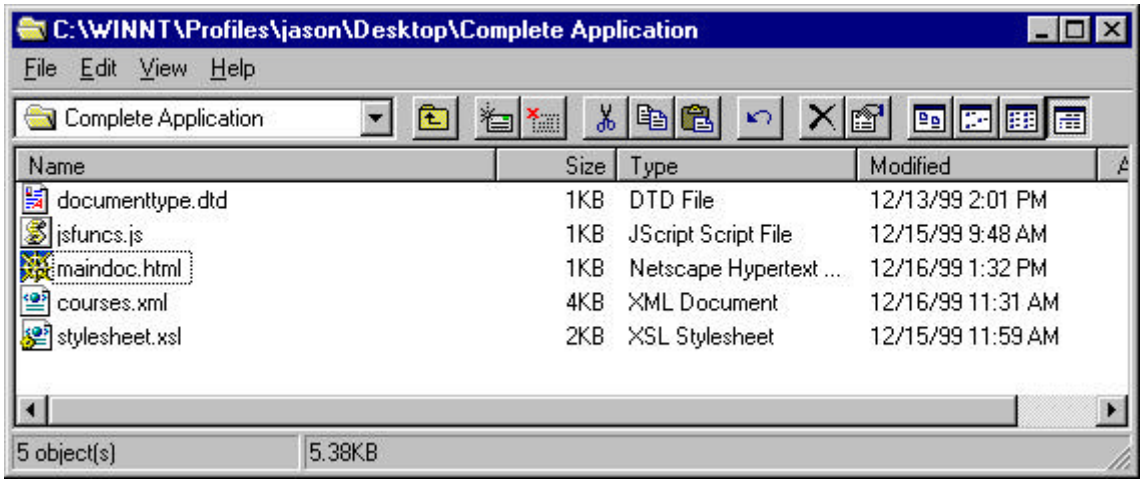



So, there's obviously something new going on here, beyond a simple HTML display. If you view the source of **maindoc.html**, you will see further evidence:

```
<html>
<head>
   <title>Displaying XML Data with XSL and the DOM</title>
<xml id="courselist" src="courses.xml"></xml>
<xml id="coursedisplay" src="stylesheet.xsl"></xml>
<script language="JavaScript" src="jsfuncs.js"></script>
</head>

<body
onLoad="dataTarget.innerHTML=courselist.transformNode(coursedisplay.XMLDocu
ment)"
bgcolor="papayawhip">

<div id="dataTarget"></div>

</body>
</html>
```

## The Components of an XML Application

As you can see, there are links to three external documents in the above code. These, along with the HTML, provide the main blocks of an XML application. We'll examine each of these briefly below, and then in more depth later. To begin, take a look at these files:




The components of a Web-based XML application are (generally) as follows:

- **An HTML File**: The HTML file acts as a wrapper for calling the other components of an XML application, and can also set certain display defaults (such as the background color in the above example). As well, it will specify areas of the page (normally through the **DIV** and **SPAN** tags) where XML data is to be displayed.

- **An XML File**: The XML file contains the data, organized hierarchically and structured internally. It is accessed in HTML with the **<XML>** tag:

  ```
  <xml id="courselist" src="courses.xml"></xml>
  ```

  and can either live internally to the HTML document or (more commonly) be stored in an external source, as in the above example. In addition, XML can be dynamically generated by a server-side process such as ASP, CGI, or ColdFusion.

- **An XSL or CSS File:** You will need a stylesheet to format the data for display in the browser. There are two options, not mutually exclusive. **C**ascading **S**tyle **S**heets (CSS) can determine the display characteristics of data in a browser. So, you can specify (for example) that your course names be **green**, your day numbers be **bold**, and everything be displayed in the font **Tahoma**.

However, you will often want to build more complex structures to display your data. These might include HTML tables, lists, borders, and more, in addition to text formatting. e**X**tensible **S**tylesheet **L**anguage (XSL) can generate HTML code, access CSS pages, and even generate other XML code! XSL files are actually XML code, with scripted extensions that give them significant power and flexibility. We will spend several exercises exploring the capabilities of XSL.

- **A DTD or Schema File:** Most XML applications require a **D**ocument **T**ype **D**efinition (DTD) or a Schema to be specified for the XML data. This DTD / Schema specifies the acceptable structure of the XML, such as the nested hierarchy, the attributes of tags, if any, and their acceptable values and data types. Both DTDs and Schemas can be applied externally, allowing several different data sheets to be validated against the same document specifics.

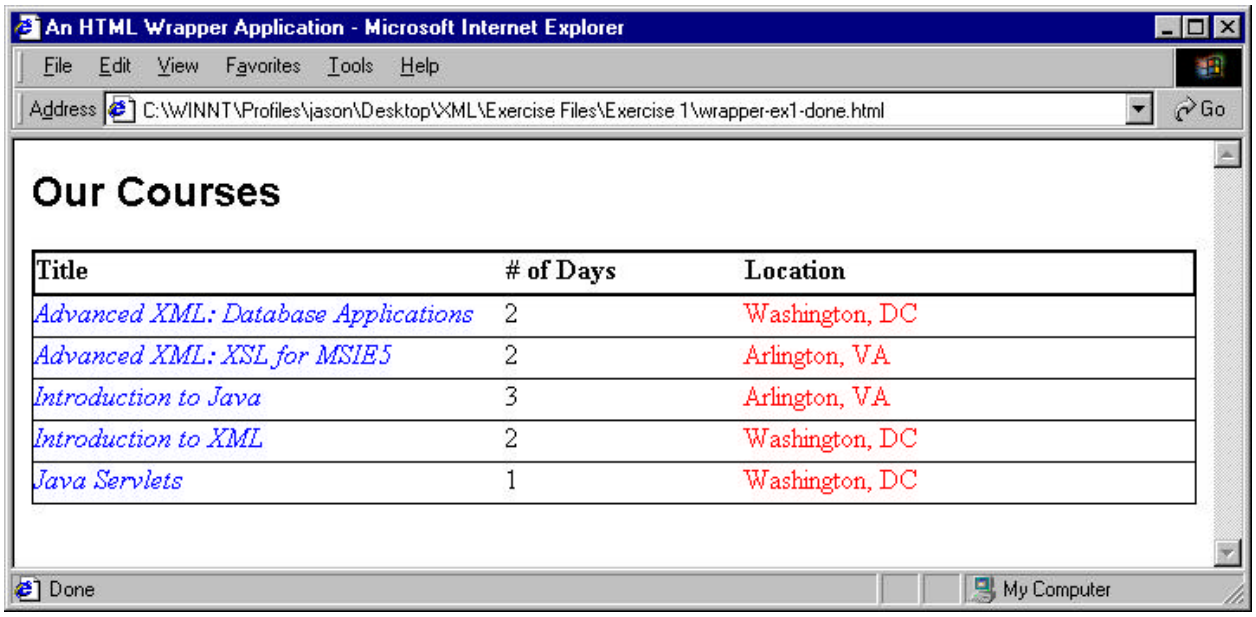
- **A JavaScript File:** (optional) Many of the most appealing features of XML (such as dynamic resorting, tree structure, and visibility changes) are accessed through XML / XSL interaction with JavaScript. It is often most convenient to write JavaScript functions as a separate **.js** file, and then make that file available to as many different applications as necessary. Of course, it is also possible to specify JavaScript in an internal `<SCRIPT>…</SCRIPT>` block, or, for simple applications, directly in-line.

We will spend significant time examining each of these components a little later in the tutorial. As a first step, you will turn a collection of XML/XSL/DTD/CSS/JS components into an XML application by finishing the HTML wrapper.

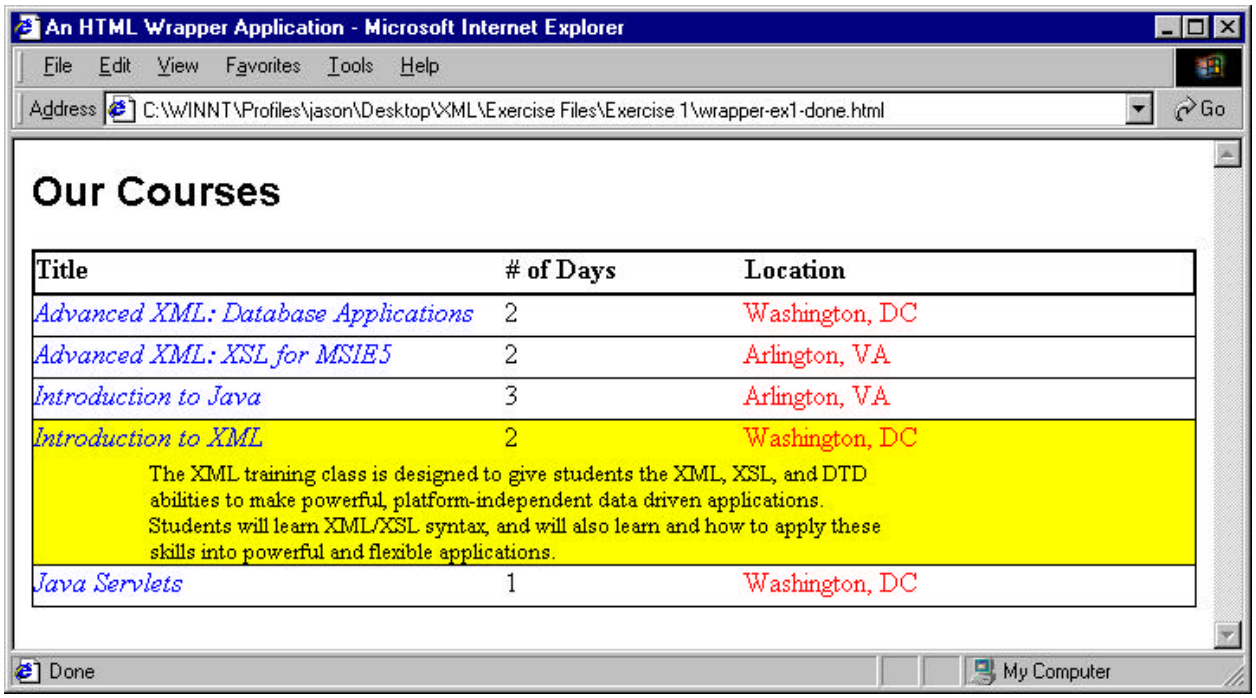
## Exercise 1: Completing an HTML "Wrapper"

In this exercise, you will be completing the HTML front-end to an XML application. The XML, XSL, and JS files are already completed for you (if you're interested, there is a DTD, but it's internal to the XML file).

When you have completed the exercise, the resulting page should look as follows:

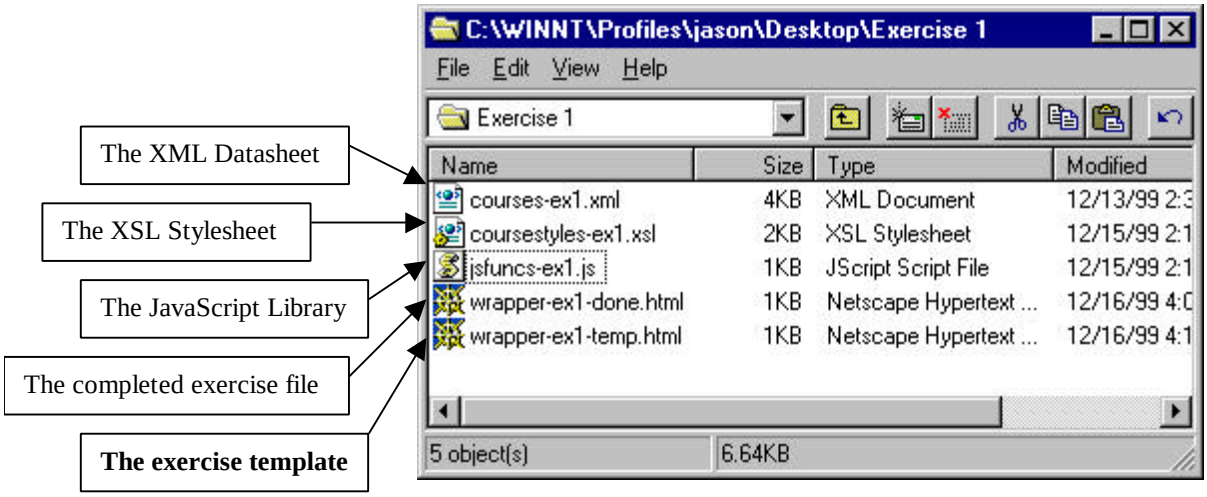



You should also be able to reorder the listing based on the different fields, trigger a mouse-over driven highlight, and make visible a display of additional data by clicking on the course title or number of days. For example, clicking on "Introduction to XML" would produce a display like the following:




To complete this exercise:

7. Take a look at the files for **exercise 1**. You should see the following files:




8. Open **wrapper-ex1-temp.html** in **Notepad** (or some other text editor). You should see the following code:

```
<html>
<head>
   <title>An HTML Wrapper Application</title>

<!-- INSERT AN <xml> TAG TO LINK TO THE XML DATA.
```

```
     THE ELEMENT id SHOULD BE "courselist" AND THE SOURCE
     SHOULD BE "courses-ex1.xml" -->
<!-- INSERT AN <xml> TAG TO LINK TO THE XSL STYLESHEET.
     THE ELEMENT id SHOULD BE "coursedisplay" AND THE
     SOURCE SHOULD BE "coursestyles-ex1.xsl" -->
<!-- INSERT A <script> TAG TO LINK TO A LIBRARY OF
     JAVASCRIPT FUNCTIONS.  THE SCRIPT language SHOULD
     BE "JavaScript" AND THE SOURCE SHOULD BE "jsfuncs-ex1.js" -->

</head>

<body
onLoad="dataTarget.innerHTML=courselist.transformNode(coursedisplay
.XMLDocument)">

<div id="dataTarget"></div>

</body>
</html>
```

9. Please complete the sections in **bold**. When you're done, save the file back into the same directory.

10. Open Internet Explorer 5, and from within IE, open **wrapper-ex1-temp.html**.

11. If you need to make corrections, edit your page in Notepad, save it, then reload it in the browser.

## If you are done early…

- Open **courses-ex1.xml** in Internet Explorer, and spend some time exploring how IE5 displays XML data.

- Open up **courses-ex1.xml** in Notepad, and take a look at the internal structure of the tags. Also, take a look at the document type definition in the head of the page. What happens if you add an XML tag?

- Open **wrapper-ex1-temp.html** in Netscape. What do you see?

## A Possible Solution to Exercise 1…

As stored in **wrapper-ex1-done.html**:

```
<html>
<head>
   <title>An HTML Wrapper Application</title>

<xml id="courselist" src="courses-ex1.xml"></xml>
<xml id="coursedisplay" src="coursestyles-ex1.xsl"></xml>
<script language="JavaScript" src="jsfuncs-ex1.js"></script>

</head>

<body
onLoad="dataTarget.innerHTML=courselist.transformNode(coursedisplay.
XMLDocument)">

<div id="dataTarget"></div>

</body>
</html>
```

# XML Syntax

XML syntax is generally very intuitive, although those of you who are used to HTML may find some of the rigorous rules of XML a bit tricky at first. However, if you understand the purposes behind the rules, they will quickly become second nature. To begin, let's take a deeper look at the example we saw earlier (**simple_example.xml**):

```
<?xml version="1.0"?>

<client>
   <name>
      <firstname>John</firstname>
      <lastname>Doe</lastname>
   </name>
   <address type="home">
      <street_address>123 School St.</street_address>
      <city>Ithaca</city>
      <state>NY</state>
      <zip>14850</zip>
   </address>
   <age>26</age>
</client>
```

The example illustrates several important rules of XML, explained below. But, before we begin, let's settle on some terminology (a complete glossary is included in **Appendix C**):

- **Tag:** An individual string of text that determines the opening or closing component of an element. In the XML example above, an example of a tag would be **address**. Tags are defined by a pair of opening and closing angle-brackets, and so can include any attributes. Note that this does not include either closing tags or any content enclosed by that tag.

- **Attribute:** A name and value pair contained in an element's opening tag (in the above example, **type="home**" would be an attribute).

- **Element:** The complete object, as defined by the opening and closing tags, any attributes, and any content, including sub-elements and text. In the above example, **client** would be an element, which would contain the sub-elements **name**, **address**, and **age**, and all their contents. A synonym for element is **node**.

## XML Logical Structure

### Each XML document must begin with a language declaration

The initial line of any XML file must be the language declaration. The **<? … ?>** brackets indicate processing instructions, so that the line is interpreted by the XML parser instead

of being treated as a generic, user-defined tag. In addition, it is strongly encouraged that the author also indicate the version of XML she is using (in this case **`version="1.0"`**). The version declaration is an example of an XML attribute, which will be discussed more thoroughly below.

Finally, an **important note**: XML tags are case sensitive. In the language declaration, the tag must be written in lower case. The XML parser is not able to understand a tag written **`<?XML VERSION="1.0"?>.`**

## Each XML document must have a single root element, which normally contains other child elements

In our previous example, the root element name is **client**. Normally, XML datasheets will include several elements (i.e. a list of 50 clients). However, as all markup languages (including XML) must have a single root, we specify a root-level tag to contain all these child elements. So, a more complete example might look as below (**simple_example_2.xml**). Particularly note the new **clients** element, which contains three individual **client** elements):

```
<?xml version="1.0"?>

<clients>
   <client>
      <name>
         <firstname>John</firstname>
         <lastname>Doe</lastname>
      </name>
      <address type="home">
         <street_address>123 School St.</street_address>
         <city>Ithaca</city>
         <state>NY</state>
         <zip>14850</zip>
      </address>
      <age>26</age>
   </client>
   <client>
      <name>
         <firstname>Jane</firstname>
         <lastname>Doe</lastname>
      </name>
      <address type="home">
         <street_address>1 Wall St.</street_address>
         <city>New York</city>
         <state>NY</state>
         <zip>10014</zip>
      </address>
      <age>22</age>
   </client>
   <client>
      <name>
         <firstname>Mary</firstname>
         <lastname>Doe</lastname>
```

```
        </name>
        <address type="business">
           <street_address>4000 Technology Way</street_addr ess>
           <city>Palo Alto</city>
           <state>CA</state>
           <zip>94301</zip>
        </address>
        <age>73</age>
     </client>
  </clients>
```

Note that if you're interested, the file is available as **simple_example_2.xml**.

## XML Physical Structure

The general rules of XML syntax are similar to those of HTML, but more rigorous. The distinctions from HTML fall into five main categories:

### Case Sensitivity

In HTML, tags are not case-sensitive. You might equally write the following possibilities:

```
<TITLE>My New Page</TITLE>
<TITLE>My New Page</title>
<title>My New Page</TITLE>
<title>My New Page</title>
<TiTlE>My New Page</tItLe>
```

However, in XML, the case of closing tags must match the case of the opening tags. So, only the first and the fourth of the above examples would be valid as XML:

```
<TITLE>My New Page</TITLE>
<title>My New Page</title>
```

Case is particularly important in the **processing instructions** (like the `<?xml version="1.0"?>` tag we saw above), where only one case is acceptable.

### Required Closing Tags

In HTML, many tags have developed with optional closing elements (such as the `</LI>`, `</P>`, `</TD>`, and other tags). In XML, all tags must have closing elements. When generating HTML code (as XSLs often do) that will require scripting the optional closing tags.

## New Syntax for "empty elements"

In HTML, there are many tags (such as the IMG and HR tags) that have no end tag, and no logical meaning for one. In XML, such tags must be indicated with a forward slash (/) before the ending angle bracket:

```
<emptyelement attr="value" />
```

## Tags must be nested properly

In HTML, nesting rules are relatively relaxed for most tags. So, you might reasonably write the following code examples:

```
<b>This sentence is really <i>important</b> and interesting</i>.
<b><i>This sentence is very heavily emphasized</b></i>.
```

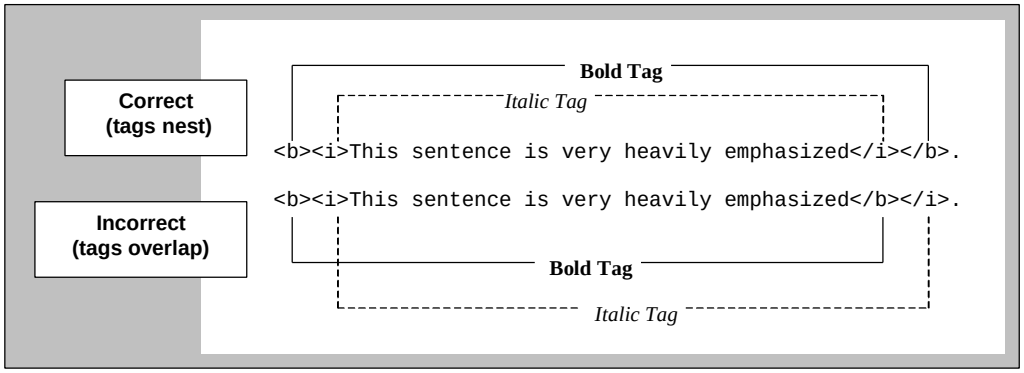
to produce the resulting:

**This sentence is really *important* *and interesting*.**
***This sentence is very heavily emphasized*.**

Neither of the above examples is well-formed, as nested tags are not completed before their parents. Both would produce errors in XML. To correct them, one could write:

```
<b>This sentence is really</b> <i><b>important</b> and
interesting</i>.
<b><i>This sentence is very heavily emphasized</i></b>.
```

Take a look at the below diagram:




## Attribute Values must be enclosed properly in single or double quotes

In HTML, you can generally get away without enclosing attribute values in quotes, except in cases where you have spaces or punctuation in the value. So, in HTML, the following tag is legal:

```
<img src=mypicture.gif name=myphoto height=80 width=150>
```

However, the above tag would not be valid in XML. In fact, all values, even numeric ones, must be enclosed in quotes. We will examine attributes in greater depth after the next exercise.

### XML Comments

Comments in XML are identical to comments in HTML. So, the following page includes a commented note:

```
<?xml version="1.0"?>
<!-- Last modified 3/23/00 -->
<clientlist>
   <client>
.
[etc.]
```

### Character References

If you wish to include non-standard characters in your document, such as copyrights (©) or Mathematical symbols (Σ) you must use their Unicode numbering system. Similarly, if you want to use a reserved character such as the angle-bracket (<)or the ampersand (&), you must use the ASCII number.

## XML Logic: Designing Datasheets

It can often be difficult to decide how to mark up your data in XML. For example, consider the following information, representing an address:

John Doe (age 26)
123 School St.
Ithaca, NY 14850

In addition to the previous example (**simple_example.xml**, and referenced on p. 17), you might imagine data marked up either less or more precisely. We have prepared examples of each, to illustrate the markup process. First the less precise example (**broad_example.xml**):

```
<?xml version="1.0"?>

<client>
   <name>John Doe</name>
   <address type="home">123 School St., Ithaca, NY, 14850</address>

   <age>26</age>
</client>
```

In the above example, we have collapsed the internal nesting of the elements **name** and **address**, producing more complex strings of text for each. By doing so, we haven't lost any textual data, but our data is less usable. We can no longer easily use individual components of name or address, as, for instance, to sort by last name would now be very difficult. Similarly, we would have a very hard time selecting only clients from New York, or even recognizing them as such.

In our next example, we have broken out our data even more precisely in the original example (**narrow_example.xml**):

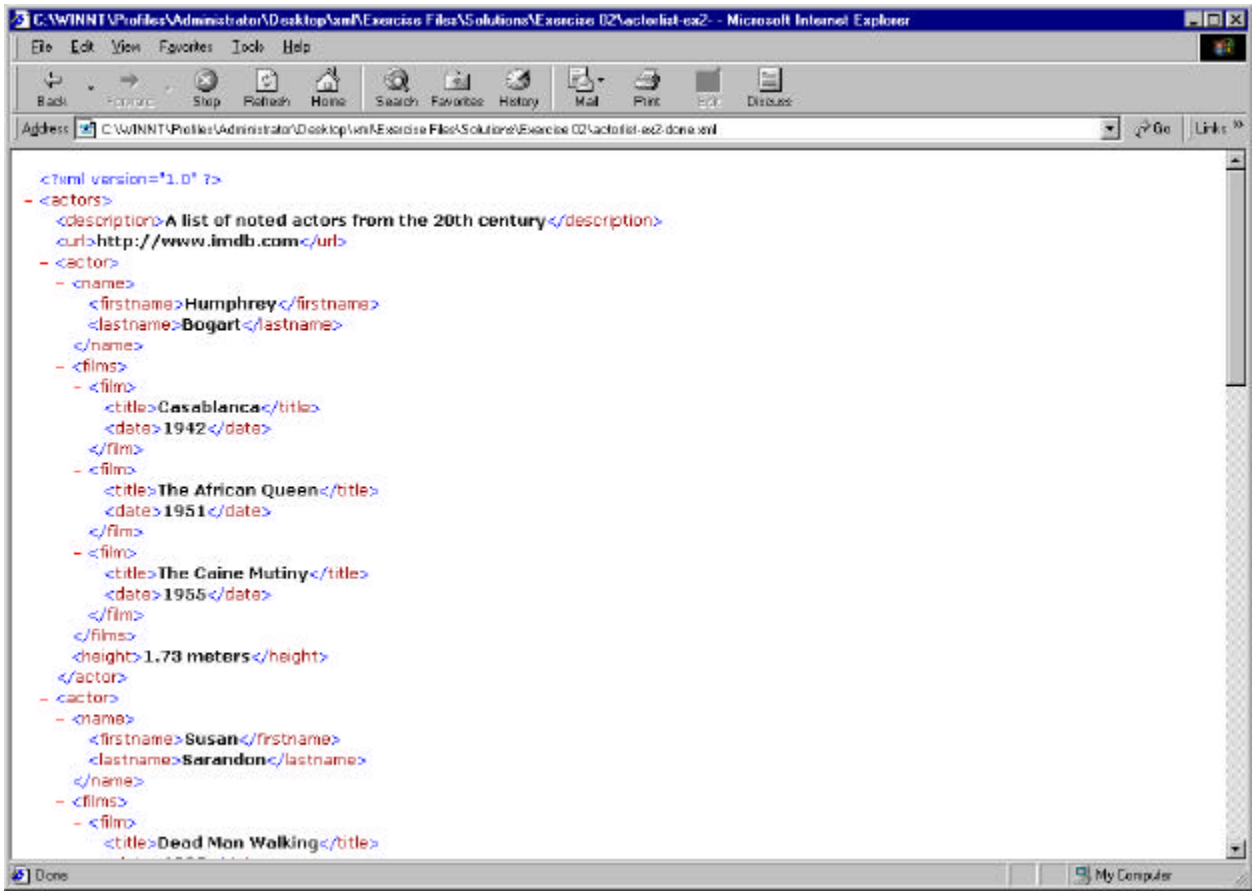
```
<?xml version="1.0"?>

<client>
   <name>
      <firstname>John</firstname>
      <lastname>Doe</lastname>
   </name>
   <address type="home">
      <street_address>
         <number>123</number>
         <street_name>School</street_name>
         <road_type>St.</road_type>
      </street_address>
      <city>Ithaca</city>
      <state>NY</state>
      <zip>14850</zip>
   </address>
   <age>26</age>
</client>
```

In our above example, we separate out the **street_address** element into constituent components. At first glance, it appears to be fine. However, what would you do with an address like "**RR1, Box 160**", or one with an apartment number? When you find that you have to do contortions to make your data fit your structure, you have probably subdivided too much.

A good rule of thumb is to *subdivide your data into logically discrete, but not arbitrary, elements*. Once you find yourself making arbitrary distinctions, it's time to stop. This may seem very fuzzy at first, but you'll soon get comfortable with the distinctions.

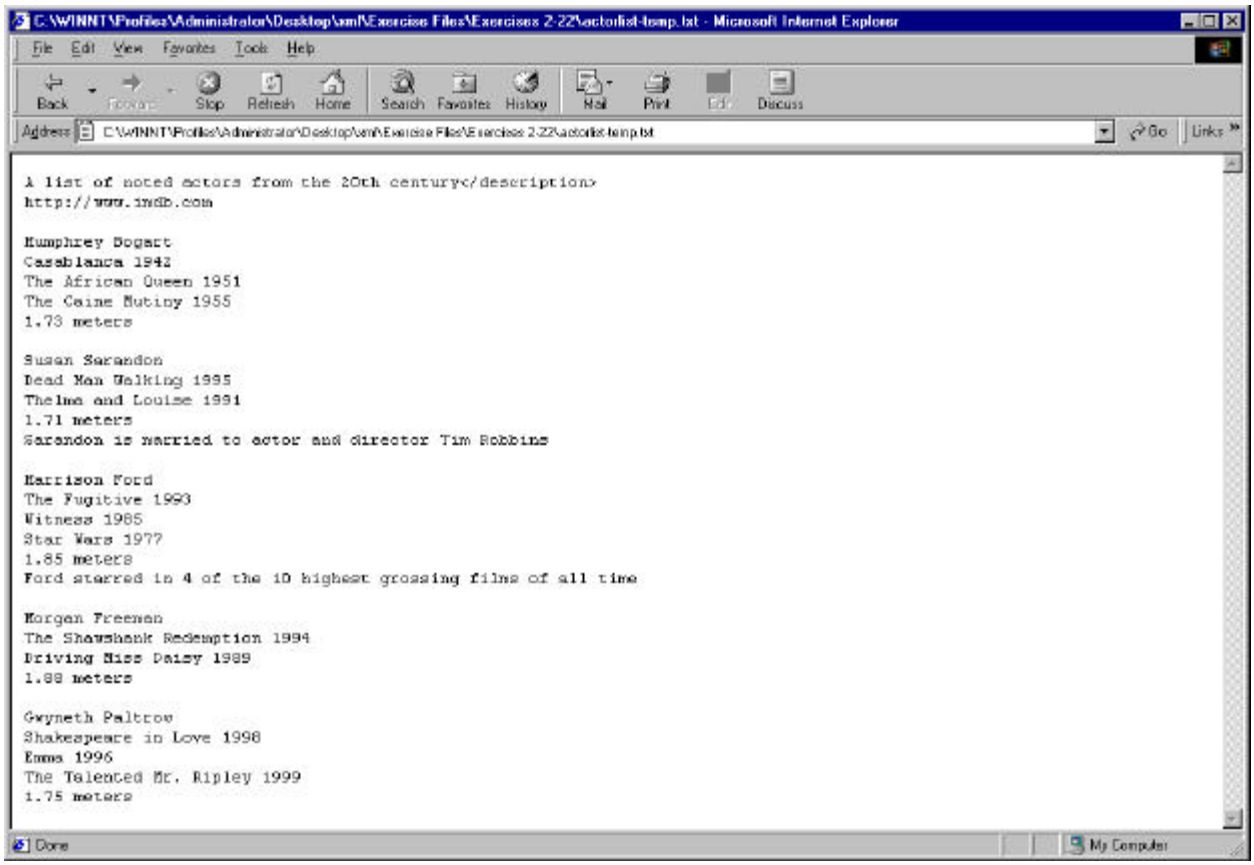
## Exercise 2: Building a well-formed XML document from text data

In this exercise, you will be taking a raw table of text data, and building an XML document out of it. When you are done, you should be able to view the document in Internet Explorer. It should look as follows (note that we are only seeing part of the data; the rest extends off the edge of the page):



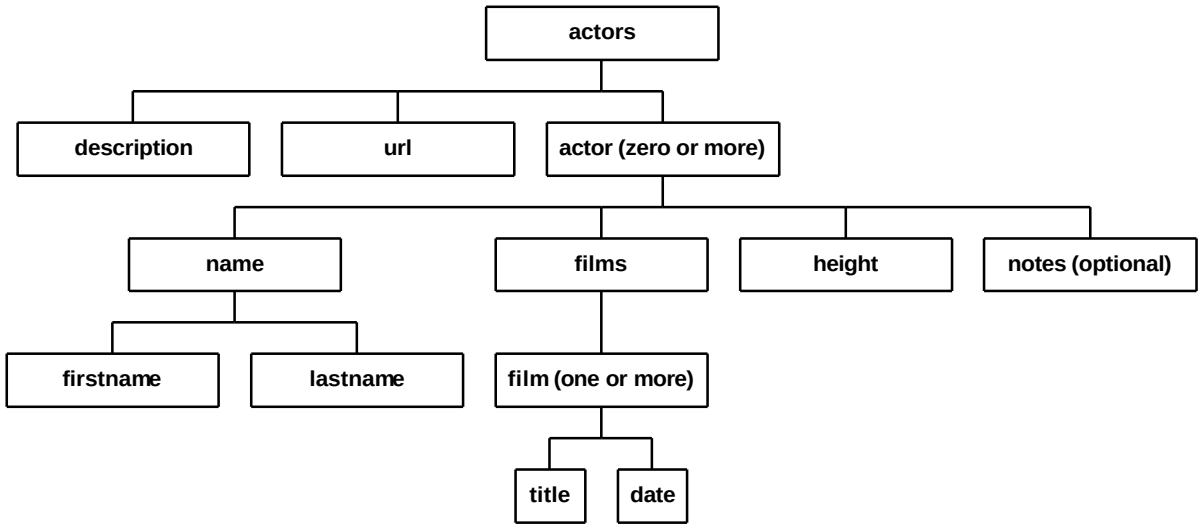

To complete this exercise:

12. Open the file **actorlist-temp.txt** in Notepad. You should see the following:




*13.* Please turn the text into an XML document. Remember that the first line will have to be the language declaration. The element hierarchy is depicted in the following diagram:

### Hierarchy of actorlist file: element and sub-elements




14. When you are done, save your file as **actorlist.xml**, and open it in Internet Explorer.
15. If you get an error message, go back into Notepad, edit your file, and reload.
16. Once you have it working, navigate up and down the hierarchy, making sure all your data is present.

## If you are done early…

- Add a listing for your favorite actor.
- Add notes data for a few of the other names in the list.

## A Possible Solution to Exercise 2

As contained in **actorlist-ex2-done.xml**:

```
<?xml version="1.0"?>

<actors>
   <description>A list of noted actors from the 20th
century</description>
   <url>http://www.imdb.com</url>
   <actor>
      <name>
         <firstname>Humphrey</firstname>
         <lastname>Bogart</lastname>
      </name>
      <films>
         <film>
            <title>Casablanca</title>
            <date>1942</date>
         </film>
         <film>
            <title>The African Queen</title>
            <date>1951</date>
         </film>
         <film>
            <title>The Caine Mutiny</title>
            <date>1955</date>
         </film>
      </films>
      <height>1.73 meters</height>
   </actor>
   <actor>
      <name>
         <firstname>Susan</firstname>
         <lastname>Sarandon</lastname>
      </name>
      <films>
         <film>
            <title>Dead Man Walking</title>
            <date>1995</date>
         </film>
         <film>
            <title>Thelma and Louise</title>
            <date>1991</date>
         </film>
      </films>
      <height>1.71 meters</height>
      <notes>Sarandon is married to actor and director Tim
Robbins</notes>
   </actor>
   <actor>
      <name>
         <firstname>Harrison</firstname>
         <lastname>Ford</lastname>
      </name>
```

```
      <films>
         <film>
            <title>The Fugitive</title>
            <date>1993</date>
         </film>
         <film>
            <title>Witness</title>
            <date>1985</date>
         </film>
         <film>
            <title>Star Wars</title>
            <date>1977</date>
         </film>
      </films>
      <height>1.85 meters</height>
      <notes>Ford starred in 4 of the 10 highest grossing films of
all time</notes>
   </actor>
   <actor>
      <name>
         <firstname>Morgan</firstname>
         <lastname>Freeman</lastname>
      </name>
      <films>
         <film>
            <title>The Shawshank Redemption</title>
            <date>1994</date>
         </film>
         <film>
            <title>Driving Miss Daisy</title>
            <date>1989</date>
         </film>
      </films>
      <height>1.88 meters</height>
   </actor>
   <actor>
      <name>
         <firstname>Gwyneth</firstname>
         <lastname>Paltrow</lastname>
      </name>
      <films>
         <film>
            <title>Shakespeare in Love</title>
            <date>1998</date>
         </film>
         <film>
            <title>Emma</title>
            <date>1996</date>
         </film>
         <film>
            <title>The Talented Mr. Ripley</title>
            <date>1999</date>
         </film>
      </films>
      <height>1.75 meters</height>
   </actor>
</actors>
```

## XML Attributes Revisited

XML attributes work similarly to HTML attributes, in that they extend the functionality of the tag that contains them. However, they are subject to a few additional syntax rules. First, XML attribute values must be surrounded by quotes. In HTML, quotes around the attribute values are generally optional, except when they contain spaces. So, the following is a valid HTML tag:

```
<img src=myfile.gif name=myimage height=80 width=150>
```

However, the above tag would not be valid in XML. All values, including numeric values, must be contained in either single or double quotes (they must match, of course). If the data is to be interpreted as numeric, it will be parsed as such in the processing script.

## Why Use Attributes?

In XML, there is little distinction made between values in attributes and values contained in sub-elements. So, you might equally well mark up your data in either of the following two ways. First, as sub-tags:

```
<car>
   <type>4-door</type>
   <color>green</color>
   <make>Ford</make>
   <model>Taurus</model>
</car>
```

or, with attributes:

```
<car type="4-door">
   <color>green</color>
   <make>Ford</make>
   <model>Taurus</model>
</car>
```

In general, you will better be able to describe your data in nested elements. However, if you are using DTDs, you have more options for validating data in attributes than you do in tag values. So, you will usually use sub-elements, except in the following 3 situations:

17. When you have a strictly limited list of possible values for a particular attribute (for example, only “yes” or “no” legally acceptable)

18. When you want to be able to specify a default value

19. When doing so allows you to get a strictly numeric value as your text. For example:

```
<weight>10 kg</weight>
```

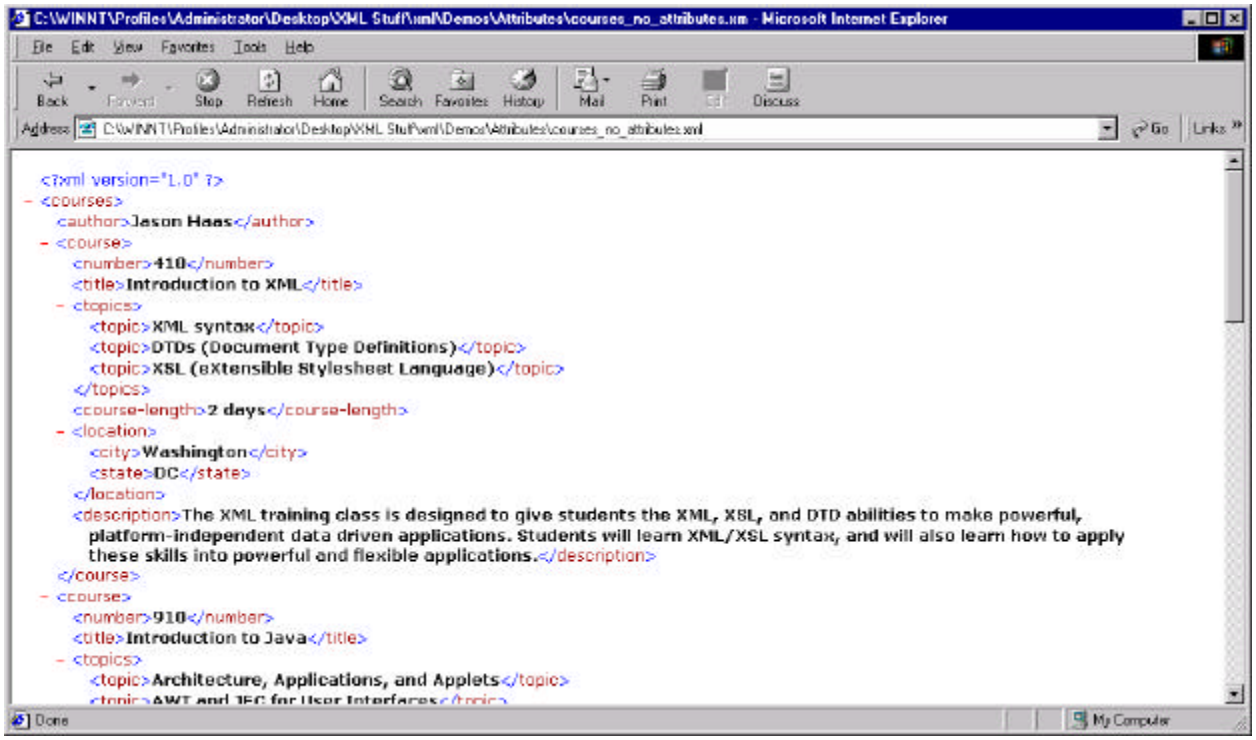
will force you to do some complicated string manipulation should you want to extract a pure numeric value out of your weight element, whereas

```
<weight units="kg">10</weight>
```

will allow you to natively treat the data as numeric (and, incidentally, will allow you easier access to the values of your units as well).

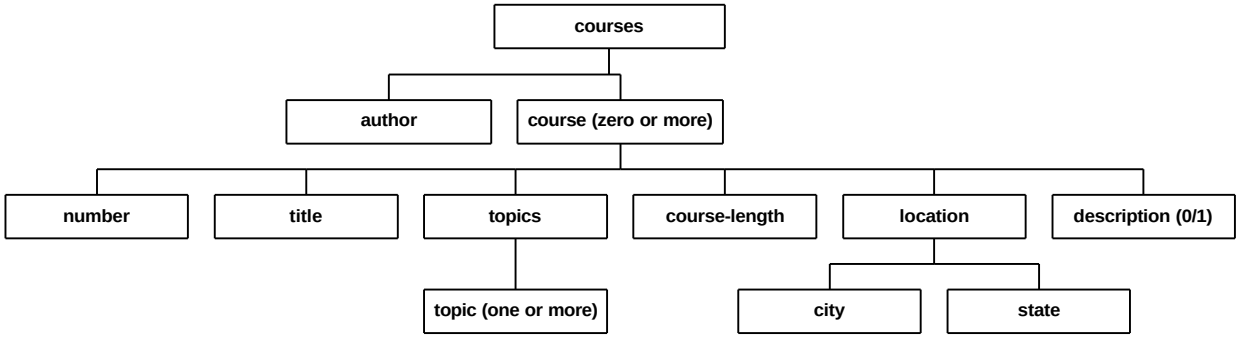
## An Introduction to Our Demo Application

We've prepared an XML application that will follow the same development steps that you do with your exercises. In each section, we will take a look at another step in the development of this application. Please first take a look at the application without any attributes (**courses_no_attributes.xml**):




Here is the XML hierarchy for the case study:

**Hierarchy of Our Case Study: the courses.xml Series**




Take a look at the code for one **course** element below. Certain portions (which make good candidates for attribute values) are highlighted in **bold**:

```
<course>
   <number>410</number>
   <title>Introduction to XML</title>
   <topics>
      <topic>XML syntax</topic>
      <topic>DTDs (Document Type Definitions)</topic>
      <topic>XSL (eXtensible Stylesheet Language)</topic>
   </topics>
   <course-length>2 days</course-length>
   <location>
      <city>Washington</city>
      <state>DC</state>
   </location>
   <description>The XML training class is designed to give
students the XML, XSL, and DTD abilities to make powerful,
platform-independent data driven applications.  Students will learn
XML/XSL syntax, and will also learn how to apply these skills into
powerful and flexible applications.</description>
</course>
```

Compare the above code to the code below, which includes attributes to provide better information about each course element, or simplifies the data for some sort of sorting:

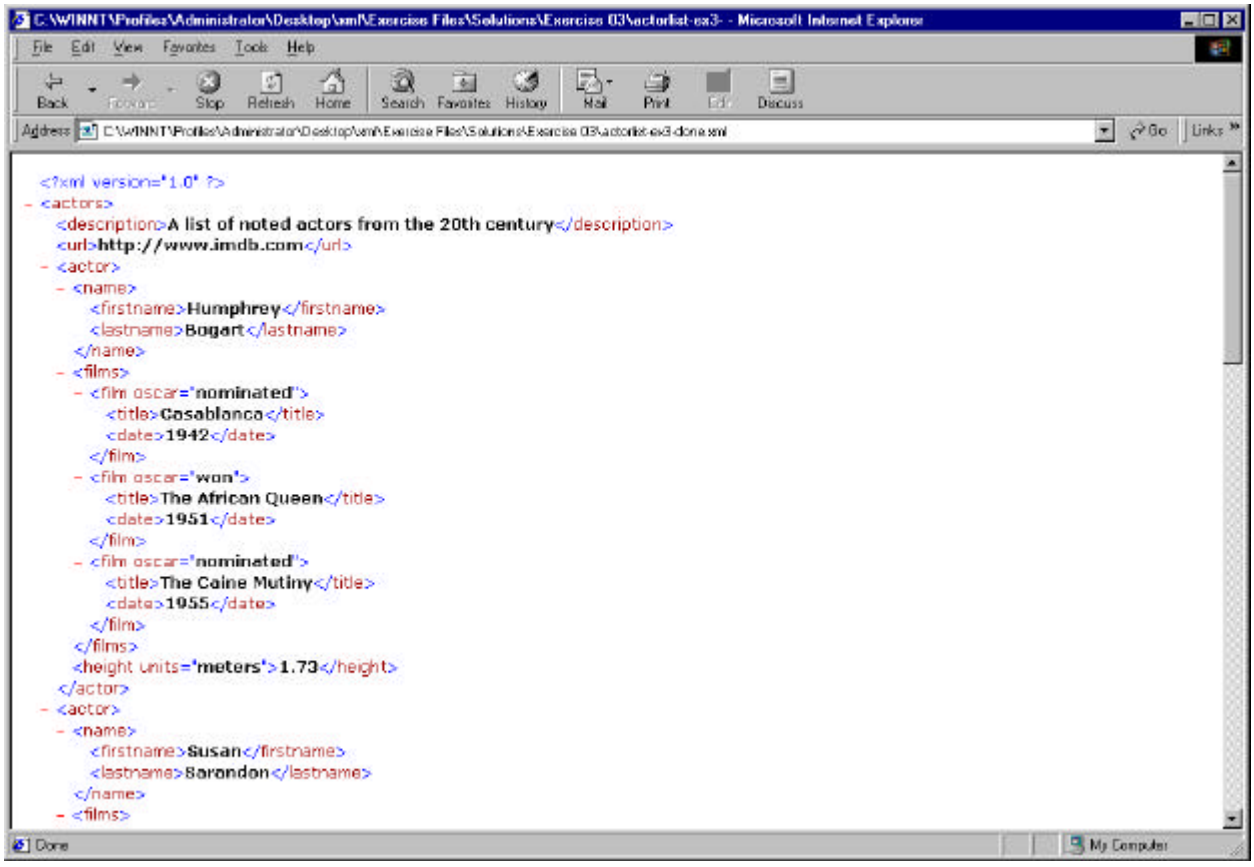
```
<course subject="XML">
   <number>410</number>
   <title>Introduction to XML</title>
   <topics>
      <topic>XML syntax</topic>
      <topic>DTDs (Document Type Definitions)</topic>
      <topic>XSL (eXtensible Stylesheet Language)</topic>
   </topics>
   <course-length scale="days">2</course-length>
   <location>
      <city>Washington</city>
      <state>DC</state>
   </location>
```

```
      <description>The XML training class is designed to give
students the XML, XSL, and DTD abilities to make powerful,
platform-independent data driven applications.  Students will learn
XML/XSL syntax, and will also learn how to apply these skills into
powerful and flexible applications.</description>
   </course>
```

In the first case, adding a course **type** attribute will allow us to sort or select on a particular course type, while in the second case, specifying the time **scale** in an attribute will allow us to treat the value of **course-length** numerically. (Note: the above code can be found as **courses_with_attributes.xml**).

## Exercise 3: Adding attributes to your XML Datasheet

In this exercise, you will be adding attributes to two tags in your copy of **actorlist.xml**. When you are done, if you view the document in Internet Explorer, it should look as follows (note again that we are only seeing part of the data; the rest extends off the edge of the page):




To complete this exercise:

20. Re-open the file **actorlist.xml** in Notepad.

21. Add the attribute "oscar" to your XML **film** tag. The possible values are "won", "nominated", and "no". If you are interested in being factually correct, the films in which the relevant actors won Oscars were:

    - The African Queen
    - Dead Man Walking
    - Shakespeare in Love

    The films for which the actors were nominated (but did not win) were:

- Casablanca
- The Caine Mutiny
- Thelma and Louise
- Witness
- The Shawshank Redemption
- Driving Miss Daisy

The films for which the actors were not nominated were:

- The Fugitive
- Star Wars
- Emma
- The Talented Mr. Ripley

22. Please add the attribute "units" to your XML **height** tag, and correct the tag's value so that it is numeric.

23. When you are done, save your file again as **actorlist.xml** and open it in Internet Explorer. If you get an error message, go back into Notepad, edit your file, and reload.

## If you are done early…

- Replace your correctly matched quotes with mismatched (or missing) ones. What is the error that is generated?

## A Possible Solution to Exercise 3

As contained in **actorlist-ex3-done.xml**:

```
<?xml version="1.0"?>

<actors>
   <description>A list of noted actors from the 20th
century</description>
   <url>http://www.imdb.com</url>
   <actor>
      <name>
         <firstname>Humphrey</firstname>
         <lastname>Bogart</lastname>
      </name>
      <films>
         <film oscar="nominated">
            <title>Casablanca</title>
            <date>1942</date>
         </film>
         <film oscar="won">
            <title>The African Queen</title>
            <date>1951</date>
         </film>
         <film oscar="nominated">
            <title>The Caine Mutiny</title>
            <date>1955</date>
         </film>
      </films>
      <height units="meters">1.73</height>
   </actor>
   <actor>
      <name>
         <firstname>Susan</firstname>
         <lastname>Sarandon</lastname>
      </name>
      <films>
         <film oscar="won">
            <title>Dead Man Walking</title>
            <date>1995</date>
         </film>
         <film oscar="nominated">
            <title>Thelma and Louise</title>
            <date>1991</date>
         </film>
      </films>
      <height units="meters">1.71</height>
      <notes>Sarandon is married to actor and director Tim
Robbins</notes>
   </actor>
   <actor>
      <name>
         <firstname>Harrison</firstname>
         <lastname>Ford</lastname>
      </name>
```

```
      <films>
         <film oscar="no">
            <title>The Fugitive</title>
            <date>1993</date>
         </film>
         <film oscar="nominated">
            <title>Witness</title>
            <date>1985</date>
         </film>
         <film oscar="no">
            <title>Star Wars</title>
            <date>1977</date>
         </film>
      </films>
      <height units="meters">1.85</height>
      <notes>Ford starred in 4 of the 10 highest grossing films of
all time</notes>
   </actor>
   <actor>
      <name>
         <firstname>Morgan</firstname>
         <lastname>Freeman</lastname>
      </name>
      <films>
         <film oscar="nominated">
            <title>The Shawshank Redemption</title>
            <date>1994</date>
         </film>
         <film oscar="nominated">
            <title>Driving Miss Daisy</title>
            <date>1989</date>
         </film>
      </films>
      <height units="meters">1.88</height>
   </actor>
   <actor>
      <name>
         <firstname>Gwyneth</firstname>
         <lastname>Paltrow</lastname>
      </name>
      <films>
         <film oscar="won">
            <title>Shakespeare in Love</title>
            <date>1998</date>
         </film>
         <film oscar="no">
            <title>Emma</title>
            <date>1996</date>
         </film>
         <film oscar="no">
            <title>The Talented Mr. Ripley</title>
            <date>1999</date>
         </film>
      </films>
      <height units="meters">1.75</height>
   </actor>
</actors>
```

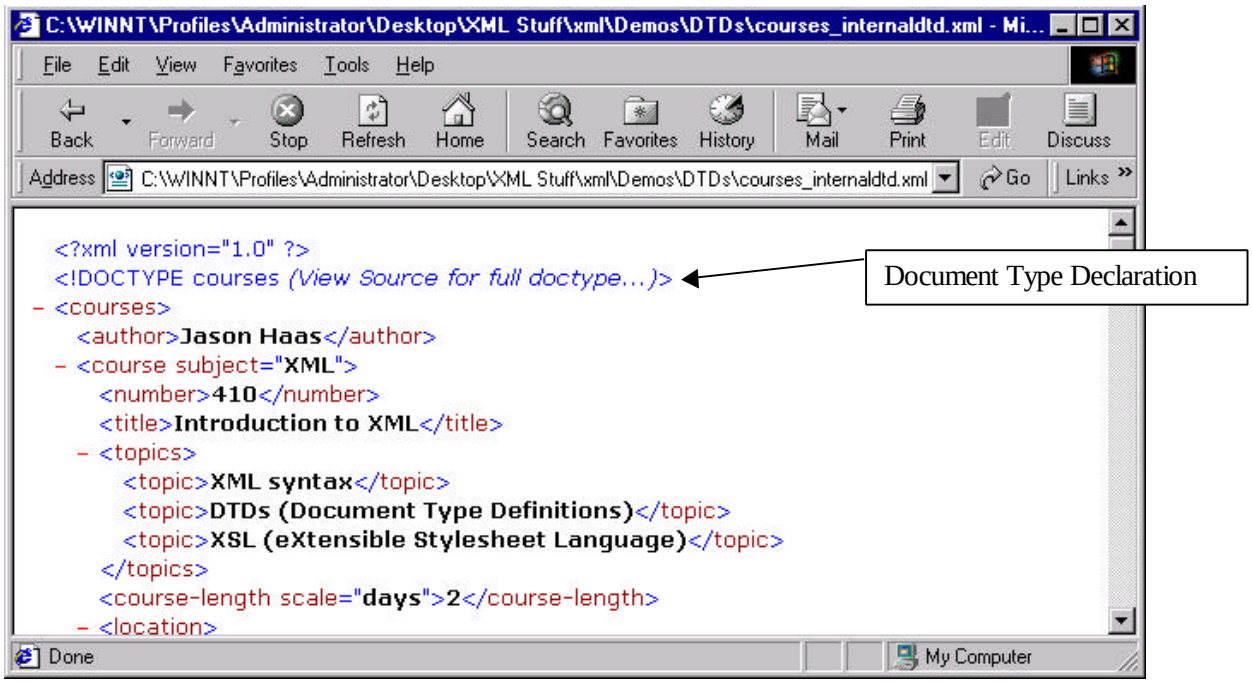
# Document Type Definitions (DTDs)

Although you can have a well-formed XML datasheet without pre-declaring its structure, many XML applications prefer to work with data that is self-validating. You make XML data self-validating by including specifications for the document type. For example, you may want users to specify the **oscar** value as a tag, rather than an attribute (and, either way, you are going to want to be clear what the appropriate structure is). A document type definition (DTD) is the specification of the structure and relationships that are valid for a given XML application. This information can be included directly at the beginning of the XML datasheet, or (more commonly) as an external document, where it is accessible by many different applications.

DTDs are particularly important for applications that use XML to exchange data between otherwise incompatible systems, applications, or organizations. Different applications can produce XML in a specified format, and applications can be developed that expect the data in the format specified. DTDs are used both to describe the specifications and to enforce them.

## Example: A Basic DTD

Take a look at the next section from our case study, found as **courses_internaldtd.xml**. When parsed, you will see one new line at the top of your document:




You may have seen a document type declaration at the beginning of some HTML documents. It is suggested that HTML authors include one at the beginning of the pages

they author, in order to identify the version of HTML that they have used. However, as the meaning of most tags is relatively constant, the document type declaration is little more than a comment on HTML version. In XML, document type declarations are crucial, as the meaning of tags is determined by the author each time a new page is designed. Let's take a look at the document type definition code from the above example:

```
<!DOCTYPE courses [
<!ELEMENT courses (author, course*)>
   <!ELEMENT author (#PCDATA)>
   <!ELEMENT course (number, title, topics, course-length, location,
description?)>
      <!ATTLIST course subject CDATA #REQUIRED>
      <!ELEMENT number (#PCDATA)>
      <!ELEMENT title (#PCDATA)>
      <!ELEMENT topics (topic+)>
         <!ELEMENT topic (#PCDATA)>
      <!ELEMENT course-length (#PCDATA)>
         <!ATTLIST course-length scale (days|hours|weeks) "days">
      <!ELEMENT location (city, state)>
         <!ELEMENT city (#PCDATA)>
         <!ELEMENT state (#PCDATA)>
      <!ELEMENT description (#PCDATA)>
]>
```

There are several new things in the above code. The most important are broken down below:

- The DOCTYPE tag, and the square brackets, must surround an internal document type definition. The syntax for the DOCTYPE tag is a bit different for an external DTD. These differences will be covered in the next section.

- In either internal or external DTDs, the DOCTYPE name must be the same as the top-level element name (in the above case, **courses**).

- Each XML element must have an ELEMENT tag. This tag will contain either the sub-elements or the data type in parentheses.

- Tags with attributes must have an ATTLIST tag specifying the possible attributes. This tag can also specify the legal values and default value for the attribute.

- There is a quantity indicating syntax specific to DTDs to specify sub-element quantities. The table below contains a summary of DTD pattern matching:

| Symbol | Meaning |
|---|---|
| + | The element must appear one or more times |
| * | The element can appear zero or more times |
| ? | The element can appear zero or one time |
| | Element must appear exactly once |

| | |
|---|---|
| \| | Separates element in a list of possible values |

- The DTD tags, including DOCTYPE, ELEMENT, and ATTLIST are all case-sensitive, and must be in upper case.

If the syntax seems a little strange, let's analyze an example of each of the four pattern-matching symbols.

## The * Symbol (Matching Zero or More Elements)

```
<!ELEMENT courses (author, course*)>
```

In the above example, we are thinking ahead to where our document type definition will be external. We want to be able to match a **courses** list in many possible circumstances. We know that most course lists will have multiple **course** elements, and we decided that you can have a valid courses list with no **course** elements (imagine a list of courses for a trainer who is on sabbatical leave), so we specify *zero or more* **course** sub-elements for each **courses** element.

## The + Symbol (Matching One or More Elements)

```
<!ELEMENT topics (topic+)>
```

Each course contains a **topics** element, which must contain at least one **topic**. However, it is perfectly acceptable for a course to contain more than one topic. So, we wish to keep out any course with no topics (would you want to take such a course?) but allow any course with *one or more* **topic** elements.

## The ? Symbol (Matching Zero or One Elements)

```
<!ELEMENT course (number, title, topics, course-length, location,
description?)>
```

Each course must contain **number**, **title**, **topics**, **course-length**, and **location**. In addition, some courses will have a **description**. To indicate an *optional* description, we follow the element name with a question mark.

## The | Symbol (Separating Possible Elements in a List)

```
<!ATTLIST course-length scale (days|hours|weeks) "days">
```

The **course-length** element has an attribute **scale**. We wish to specify acceptable values for the scale attribute, and have decided that valid options are days, hours, and weeks (so, no courses measured in centuries or minutes). To specify a list of possible options, we

follow the attribute name with a list of the options, in parentheses, separated by the pipe (vertical line) symbol. If you're looking for it on your keyboard, it's **shift-**.

### Default Values, #REQUIRED and #IMPLIED

In your DTD, you may provide a default value for an attribute. If the XML datasheet does not specify a value, the default will be used. In the above example, we are specifying "days" as the default value. If you choose not to give a default, you must specify either that the attribute is required (#REQUIRED) or optional (#IMPLIED). A third option, #FIXED, is rarely used.

### CDATA and PCDATA

When you specify a node that contains text content (either tag content or attribute values) you must specify a data type. Your two choices are **CDATA** (short for character data) and **PCDATA** (short for parsed character data). The main distinction between the two is that data defined as PCDATA will be parsed for character equivalents such as < and &.

Effectively, this means that **CDATA** is the only commonly accepted data type for attributes. For node data, **PCDATA** is much more common.

## Validating Against your DTDs

Unfortunately, the W3C standards do not declare what should happen if data does not validate successfully against its DTD. Therefore, individual implementations have varied. In Internet Explorer, although it will parse the DTD for syntax problems, default values, and the like, DTD patterns are not *evaluated against the XML datasheet* by default. It is validated if the XML data is being accessed by an outside application (such as an HTML page), but you would like to validate more flexibly. Short of building an extraneous HTML wrapper page just to check for validation against a DTD, you have three options for checking your XML:

24. You can use a DOM method **validate()** to check an XML datasheet against its DTD. Although the DOM is outside the scope of this course, we have included a utility file named **wrapper_validate.html** (which references a JavaScript function) that will use the DOM to check any XML document against its DTD.

    You can install a utility from Microsoft that will allow you to right-click in an Internet Explorer window and choose "validate document". This will validate the XML against its DTD. The utility is located at:

    - *http://msdn.microsoft.com/msdn-files/027/000/543/iexmltls.exe*

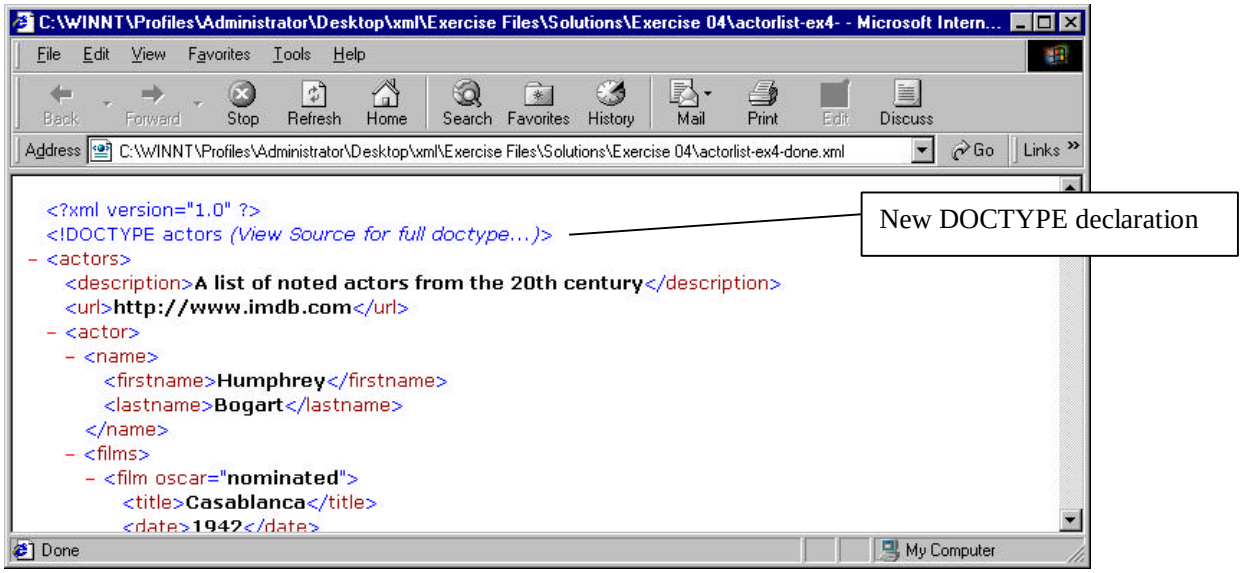
    You can use one of many DTD validation utilities available publicly on the Web. Two of the best that we have found are:

- **Brown University Scholarly Technology Group**
  (*http://www.stg.brown.edu/service/xmlvalid/*)

- **MSDN Client-Side Validator**
  (*http://msdn.microsoft.com/downloads/samples/internet/xml/xml_validator/*)

## Exercise 4: Adding an Internal DTD to your XML Datasheet

In this exercise, you will be adding a document type definition to **actorlist.xml**. When you are done, if you view the document in Internet Explorer, it you should see the following extra line in your code:




To complete this exercise:

25. Re-open the file **actorlist.xml** in Notepad.

26. Please add a document type declaration to your code, according to the following specifications:

    - The DOCTYPE name should be **actors**

    - You should accept a **description**, a **url**, and zero or more **actor** sub-elements for **actors.**

    - The **actor** element should have the required sub-elements **name**, **films**, and **height**, and the optional sub-element **notes**.

    - The **name** element should have the required sub-elements **firstname** and **lastname**.

- The **films** element should have the one or more **film** elements as children.
- The **film** element should have the attribute **oscar**, with the possible values "won", "nominated", and "no". The default value should be "no".
- The **film** element should have the required child elements **title** and **date**
- The **height** element should have the attribute **units**, which should have the possible legal values of **meters**, **cm**, and **feet**. It should be required.
- All elements should take the text type **#PCDATA**.

27. When you are done, save your file again as **actorlist.xml** and open it in Internet Explorer.
28. If you get an error message, go back into Notepad, edit your file, and reload.
29. Once you have it working, open the file **wrapper_validate.html**. If you receive the results "Document Validates OK" then you've been successful. If not, fix the error that is described.

## If you are done early…

- What happens now if a user enters "Yes" or "NO" as values for preferred? What could you do to keep the values above from producing errors?

## A Possible Solution to Exercise 4

As contained in **actorlist-ex4-done.xml**:

```
<?xml version="1.0"?>

<!DOCTYPE actors [
<!ELEMENT actors (description, url, actor*)>
   <!ELEMENT description (#PCDATA)>
   <!ELEMENT url (#PCDATA)>
   <!ELEMENT actor (name, films, height, notes?)>
      <!ELEMENT name (firstname, lastname)>
         <!ELEMENT firstname (#PCDATA)>
         <!ELEMENT lastname (#PCDATA)>
      <!ELEMENT films (film+)>
         <!ELEMENT film (title, date)>
            <!ATTLIST film oscar (nominated|won|no ) "no">
            <!ELEMENT title (#PCDATA)>
            <!ELEMENT date (#PCDATA)>
      <!ELEMENT height (#PCDATA)>
         <!ATTLIST height units (meters|cm|feet) #REQUIRED>
      <!ELEMENT notes (#PCDATA)>
]>

<actors>
   <description>A list of noted actors from the 20th
century</description>
   <url>http://www.imdb.com</url>
   <actor>
      <name>
         <firstname>Humphrey</firstname>
         <lastname>Bogart</lastname>
      </name>
      <films>
         <film oscar="nominated">
            <title>Casablanca</title>
            <date>1942</date>
         </film>
         <film oscar="won">
            <title>The African Queen</title>
            <date>1951</date>
         </film>
         <film oscar="nominated">
            <title>The Caine Mutiny</title>
            <date>1955</date>
         </film>
      </films>
      <height units="meters">1.73</height>
   </actor>
   <actor>
      <name>
         <firstname>Susan</firstname>
         <lastname>Sarandon</lastname>
      </name>
```

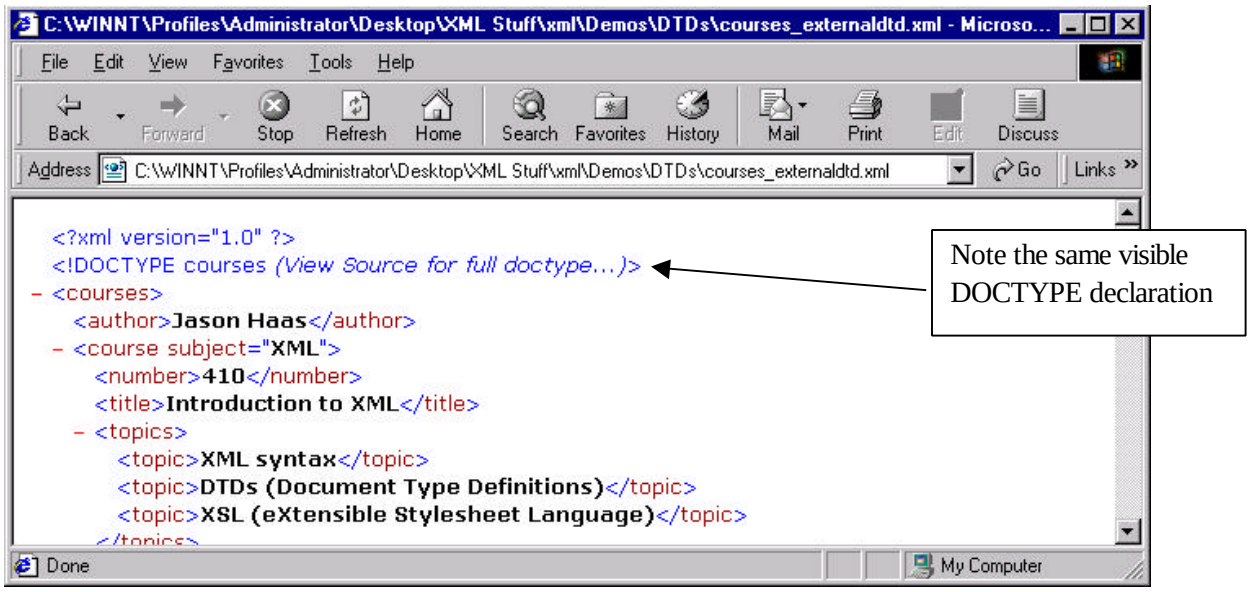
```
      <films>
         <film oscar="won">
            <title>Dead Man Walking</title>
            <date>1995</date>
         </film>
         <film oscar="nominated">
            <title>Thelma and Louise</title>
            <date>1991</date>
         </film>
      </films>
      <height units="meters">1.71</height>
      <notes>Sarandon is married to actor and director Tim
Robbins</notes>
   </actor>
   <actor>
      <name>
         <firstname>Harrison</firstname>
         <lastname>Ford</lastname>
      </name>
      <films>
         <film oscar="no">
            <title>The Fugitive</title>
            <date>1993</date>
         </film>
         <film oscar="nominated">
            <title>Witness</title>
            <date>1985</date>
         </film>
         <film oscar="no">
            <title>Star Wars</title>
            <date>1977</date>
         </film>
      </films>
      <height units="meters">1.85</height>
      <notes>Ford starred in 4 of the 10 highest grossing films of
all time</notes>
   </actor>
   <actor>
      <name>
         <firstname>Morgan</firstname>
         <lastname>Freeman</lastname>
      </name>
      <films>
         <film oscar="nominated">
            <title>The Shawshank Redemption</title>
            <date>1994</date>
         </film>
         <film oscar="nominated">
            <title>Driving Miss Daisy</title>
            <date>1989</date>
         </film>
      </films>
      <height units="meters">1.88</height>
   </actor>
   <actor>
      <name>
         <firstname>Gwyneth</firstname>
```

```
            <lastname>Paltrow</lastname>
         </name>
         <films>
            <film oscar="won">
               <title>Shakespeare in Love</title>
               <date>1998</date>
            </film>
            <film oscar="no">
               <title>Emma</title>
               <date>1996</date>
            </film>
            <film oscar="no">
               <title>The Talented Mr. Ripley</title>
               <date>1999</date>
            </film>
         </films>
         <height units="meters">1.75</height>
      </actor>
   </actors>
```

## External DTDs

DTDs can be (and more typically, are) specified externally, where they can be accessed by many XML datasheets, or distributed to different people or applications to ensure uniformity. Using external DTDs is an excellent, scalable way of ensuring that individual datasheets (which can be produced by many different people and applications) all follow the necessary structure.

The tag to link to an external DTD is relatively simple. Again, we're taking a look at the ongoing demo (this example can be found as **courses_externaldtd.xml**):




As you can see above, the browser treats an external DTD the same as an internal DTD, as what it references and parses is the DOCTYPE name. The actual code structure is below:

```
<?xml version="1.0"?>

<!DOCTYPE courses SYSTEM "courses.dtd">

<courses>
   <author>Jason Haas</author>
   <course subject="XML">
      <title>Introduction to XML</title>
      <topics>
         <topic>XML syntax</topic>
.
.
[code deleted]
.
.
```

Inside the DTD itself, the code is unchanged (except that there is no DOCTYPE declaration, as the DOCTYPE itself is declared in the XML document). From **courses.dtd**:

```
<!ELEMENT courses (author, course*)>
   <!ELEMENT author (#PCDATA)>
   <!ELEMENT course (number, title, topics, course-length,
location, description?)>
   <!ATTLIST course subject CDATA #REQUIRED>
   <!ELEMENT number (#PCDATA)>
   <!ELEMENT title (#PCDATA)>
   <!ELEMENT topics (topic+)>
      <!ELEMENT topic (#PCDATA)>
   <!ELEMENT course-length (#PCDATA)>
   <!ATTLIST course-length scale (days|hours|weeks) "days">
   <!ELEMENT location (city, state)>
      <!ELEMENT city (#PCDATA)>
      <!ELEMENT state (#PCDATA)>
   <!ELEMENT description (#PCDATA)>
```

As you can see, there is no special punctuation necessary to begin a DTD. It is not an XML file, and needs (in fact, can take) no `<?xml?>` language declaration. It also does not need to be contained inside the square brackets of the DOCTYPE declaration.

One caution: you will still need to make sure that your DOCTYPE name matches the name of your primary element. It can be harder to notice with an external DTD that your names don't match, and you will need to pay special care that they do.

## Public vs. System DTDs

You have two options for external DTDs. Their syntax is given below:

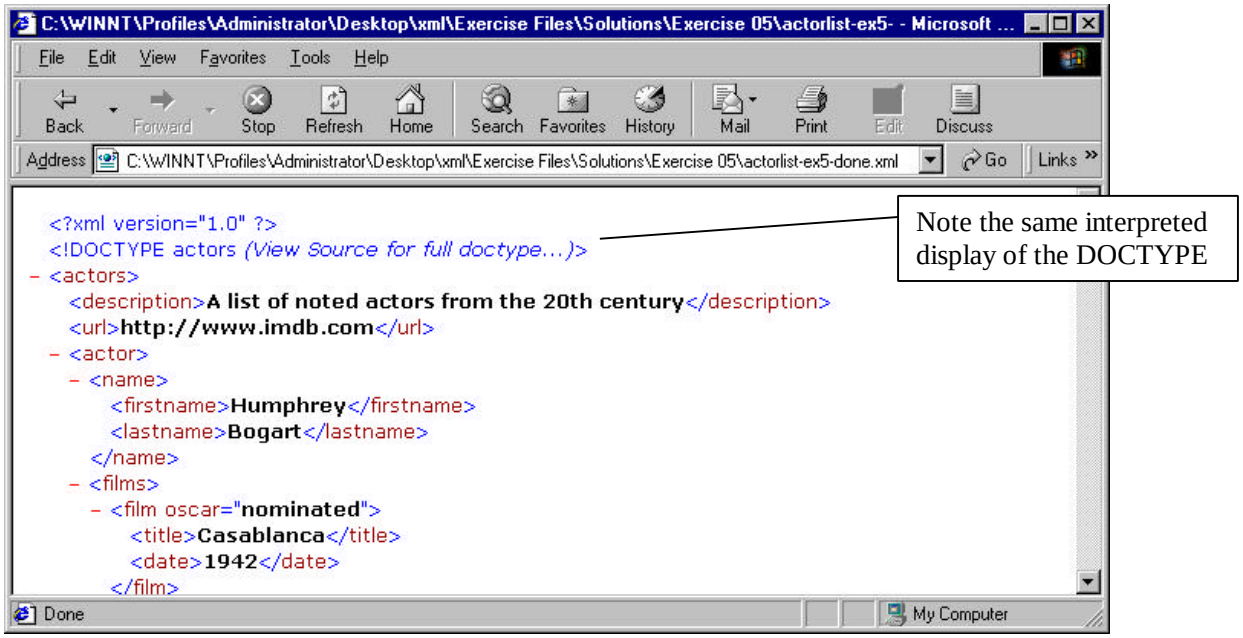
```
<!DOCTYPE docname SYSTEM "path_to_DTD_file">
<!DOCTYPE docname PUBLIC "public_identifier" "path_to_DTD_file">
```

An external SYSTEM DTD allows you to specify the document type name, and a file (or path to a file) that will serve as the DTD. A PUBLIC DTD allows you to specify an internal or external location or library of DTDs. If the file cannot be found there (or the library cannot be located) the file or path is used. The more commonly used of the two is SYSTEM, as the library of existing XML document types is currently limited. As XML applications become more common, it is likely that PUBLIC DTDs will come into broader use.

Note that there are no operators in a DOCTYPE tag (no equal signs, for example). The tag is parsed as individual elements, separated by a space. Be careful to maintain your spacing in DTD statements.

## Exercise 5: Creating an External DTD, and linking it to your XML Datasheet

In this exercise, you will be moving the DTD you created in the previous exercise to an external file, and then modifying your XML datasheet to link to it as a SYSTEM DTD. When you are done, the display should be no different from what it was at the end of the previous exercise:




To complete this exercise:

30. Re-open the file **actorlist.xml** in Notepad.

31. Open a blank document in Notepad, and transfer the DTD code from **actorlist.xml** to the new file.

32. Save the new file as **actordtd.dtd**.

33. Modify **actorlist.xml** so that it contains a link to **actordtd.dtd**. The link should be in the form of a SYSTEM DTD.

34. When you are done, save your XML file again as **actorlist.xml** and open it in Internet Explorer.

35. If you get an error message, go back into Notepad, edit your file, and reload.

36. Once you have it working, check the utility **wrapper_validate.html**, and make sure that the validation is successful.

## A Possible Solution to Exercise 5

As contained in **actorlist-ex5-done.xml**:

```
<?xml version="1.0"?>

<!DOCTYPE actors SYSTEM "actordtd-ex5-done.dtd">

<actors>
   <description>A list of noted actors from the 20th
century</description>
   <url>http://www.imdb.com</url>
   <actor>
      <name>
         <firstname>Humphrey</firstname>
         <lastname>Bogart</lastname>
      </name>
      <films>
         <film oscar="nominated">
            <title>Casablanca</title>
            <date>1942</date>
         </film>
         <film oscar="won">
            <title>The African Queen</title>
            <date>1951</date>
         </film>
         <film oscar="nominated">
            <title>The Caine Mutiny</title>
            <date>1955</date>
         </film>
      </films>
      <height units="meters">1.73</height>
   </actor>
   <actor>
      <name>
         <firstname>Susan</firstname>
         <lastname>Sarandon</lastname>
      </name>
      <films>
         <film oscar="won">
            <title>Dead Man Walking</title>
            <date>1995</date>
         </film>
         <film oscar="nominated">
            <title>Thelma and Louise</title>
            <date>1991</date>
         </film>
      </films>
      <height units="meters">1.71</height>
      <notes>Sarandon is married to actor and director Tim
Robbins</notes>
   </actor>
   <actor>
```

```
      <name>
         <firstname>Harrison</firstname>
         <lastname>Ford</lastname>
      </name>
      <films>
         <film oscar="no">
            <title>The Fugitive</title>
            <date>1993</date>
         </film>
         <film oscar="nominated">
            <title>Witness</title>
            <date>1985</date>
         </film>
         <film oscar="no">
            <title>Star Wars</title>
            <date>1977</date>
         </film>
      </films>
      <height units="meters">1.85</height>
      <notes>Ford starred in 4 of the 10 highest grossing films of
all time</notes>
   </actor>
   <actor>
      <name>
         <firstname>Morgan</firstname>
         <lastname>Freeman</lastname>
      </name>
      <films>
         <film oscar="nominated">
            <title>The Shawshank Redemption</title>
            <date>1994</date>
         </film>
         <film oscar="nominated">
            <title>Driving Miss Daisy</title>
            <date>1989</date>
         </film>
      </films>
      <height units="meters">1.88</height>
   </actor>
   <actor>
      <name>
         <firstname>Gwyneth</firstname>
         <lastname>Paltrow</lastname>
      </name>
      <films>
         <film oscar="won">
            <title>Shakespeare in Love</title>
            <date>1998</date>
         </film>
         <film oscar="no">
            <title>Emma</title>
            <date>1996</date>
         </film>
         <film oscar="no">
            <title>The Talented Mr. Ripley</title>
            <date>1999</date>
         </film>
```

```
      </films>
      <height units="meters">1.75</height>
   </actor>
</actors>
```

As contained in **actordtd-ex5-done.dtd**:

```
<!ELEMENT actors (description, url, actor*)>
   <!ELEMENT description (#PCDATA)>
   <!ELEMENT url (#PCDATA)>
   <!ELEMENT actor (name, films, height, notes?)>
      <!ELEMENT name (firstname, lastname)>
         <!ELEMENT firstname (#PCDATA)>
         <!ELEMENT lastname (#PCDATA)>
      <!ELEMENT films (film+)>
         <!ELEMENT film (title, date)>
            <!ATTLIST film oscar (nominated|won|no) "no">
            <!ELEMENT title (#PCDATA)>
            <!ELEMENT date (#PCDATA)>
      <!ELEMENT height (#PCDATA)>
         <!ATTLIST height units (meters|cm|feet) #REQUIRED>
      <!ELEMENT notes (#PCDATA)>
```

## Here are the top 5 reasons to choose WestLake's classroom-based, hands-on method of instruction:

**#5: Web Development Skills are the #1 need of employers today**
We have the skills you need to compete in the 21st century! We guide you all the way from Beginning HTML to the cutting edge technologies like XML, ASP, and Wireless Markup Language. We do it all.

**#4: Our courseware makes learning and retaining the material enjoyable and easy**
Written by our seasoned training staff, our courseware contains the complete instructional content of the class, making it easy to review later what you have learned. And because WestLake is vendor-neutral, you will always get a balanced perspective on each technology.

**#3: Service you receive – before, during, and after class**
Before class, our expert registration staff will help you select the class that's right for you. During class, you will be served food and drink, and be encouraged to ask questions. After class, support is available from trainers seven days per week via our online forums.

**#2: Small Class Sizes**
We limit our class size to 12 students. Every student works on a state of the art PC with high speed Internet access. In addition, there is a monitor next to your desk where you can see what is on the instructor's screen – no more straining to see the difference between a colon and a semi-colon at 50 paces.

**And the #1 reason students select WestLake as the Web Development Training Leader:**
Students leave prepared to immediately apply what they have learned. Rather than sitting through a boring lecture, you will spend a large portion of class time doing hands-on development with the technology you are learning. Leave prepared to immediately tackle your projects!

If you have any questions regarding our classes, please email info@westlake.com or call us toll-free at 866.WESTLAKE (866.937.8525) and select option 1. You can register at the same number or online at http://www.westlake.com/register/.

To have WestLake bring a class to you, please email customized@westlake.com or call us toll-free at 866.WESTLAKE (866.937.8525) and select option 2.